



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ
ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΤΕΧΝΗΤΗ ΝΟΥΜΟΣΗΝΗ

Ακαδημαϊκό Έτος 2023-2024

ΤΕΧΝΙΚΗ ΑΝΑΦΟΡΑ

ΤΕΧΝΗΤΗ ΝΟΥΜΟΣΗΝΗ

ΕΡΓΑΣΤΗΡΙΟ

ΑΣΚΗΣΗ 1

ΚΙΟΥΡΤ-ΜΕΧΜΕΤΑΛΛΗ ΑΧΜΕΤ

Α.Μ.: 1084672

Έτος Φοίτησης: 4ο

Πάτρα 2023-24

Μέρος Α: Παζλ 8 πλακιδίων

1)

Αλγόριθμος	Μήκος Λύσης	Είναι βέλτιστη;	Κόστος
BFS	12	ΝΑΙ	1323
DFS	>1k	ΟΧΙ	>3720
IDS**	>16	ΟΧΙ	>792*
Greedy, Euclidean	12	ΝΑΙ	13
Greedy, Manhattan	36	ΟΧΙ	227
Greedy, out-of-place	28	ΟΧΙ	323
A*, Euclidean	12	ΝΑΙ	43
A*, Manhattan	12	ΝΑΙ	26
A*, out-of-place	12	ΝΑΙ	100

*Στην 16^η επανάληψη είχαμε το συγκεκριμένο κόστος

Μετά την εκτέλεση των αλγόριθμων ανά παραπάνω από 5 λεπτά το DFS και το IDS δεν τερματίζουν.

** Ο IDS που αναζητά σε δύο επίπεδα, θα περιμέναμε να έχουμε μία βέλτιστη λύση, αφού με άλλα παραδείγματα δεν κολλάει και βρίσκει ακόμα και την βέλτιστή λύση. Στην δική μας περίπτωση δεν μας τερματίζει επειδή η λύση είναι πάρα πολύ μεγάλη. Βέβαια γνωρίζουμε από την θεωρία ότι από την στιγμή που έχουμε συγκεκριμένα δεδομένα το IDS τερματίζει. Άρα ξέρουμε ότι και το πρόβλημα μας θα τερματίσει κάποτε θεωρητικά.

2) Όπως μπορούμε να παρατηρήσουμε και στο παραπάνω πίνακα βλέπουμε ότι τα DFS, IDS, Greedy-Manhattan, Greedy-out-of-place, δεν έχουν μία βέλτιστη λύση για το Α μέρος της εργασίας.

Ας εξηγήσουμε για κάθε αλγόριθμο τι συμβαίνει και δεν έχει μία βέλτιστη λύση.

α) Depth-First Search (DFS)

ο DFS εξερευνά τον χώρο αναζήτησης με τρόπο πρώτου βάθους, πράγμα που σημαίνει ότι δίνει προτεραιότητα στο να πηγαίνει όσο το δυνατόν πιο βαθιά σε έναν κλάδο του δέντρου αναζήτησης πριν κάνει πίσω για να εξερευνήσει άλλους κλάδους. Στο πλαίσιο του Παζλ 8, αυτό σημαίνει ότι μπορεί να εξερευνήσει εκτενώς έναν κλάδο του δέντρου αναζήτησης πριν εξετάσει άλλες επιλογές.

Επειδή, η προτεραιότητα που παρέχει είναι το βάθος, συγκεντρώνεται μόνο για το βάθος του προβλήματος.

β) Iterative Deepening Search (IDS)

Το IDS εξερευνά αυτό το χώρο κατάστασης σταδιακά, ξεκινώντας με βάθος 0 και αυξάνοντάς το σταδιακά.

Το IDS λειτουργεί χωρίς να λαμβάνεται υπόψη το κόστος που σχετίζεται με κάθε μετάβαση κατάστασης. Εξερευνά τον χώρο αναζήτησης αποκλειστικά με βάση τα επίπεδα βάθους και

δεν λαμβάνει υπόψη τον αριθμό των κινήσεων ή το κόστος που απαιτείται για την επίτευξη μιας συγκεκριμένης κατάστασης. Αυτό μπορεί να οδηγήσει στο IDS να εξερευνήσει αναποτελεσματικά μονοπάτια που απαιτούν περισσότερες κινήσεις για να φτάσει στην κατάσταση στόχου πριν ανακαλύψει τη βέλτιστη λύση.

Τέλος, εξερευνά επανειλημμένα τις ίδιες καταστάσεις σε διαφορετικά επίπεδα βάθους, πιθανώς να επισκέπτεται ξανά καταστάσεις πολλές φορές. Που αυτό οδηγεί σε περιττή εξερεύνηση και περιττή εργασία, ειδικά όταν η βέλτιστη λύση βρίσκεται σε μικρότερο βάθος.

γ) Greedy, Manhattan

Ο αλγόριθμος Greedy έχει μία συμπεριφορά που επιλέγει την καλύτερη επιλογή διαθέσιμη εκείνη την στιγμή που δεν σκέφτεται προηγούμενες επιλογές. Αλλά με την ευρετική Manhattan δεν καταφέρνει να βρει την βέλτιστη λύση.

Η ευρετική απόσταση του Manhattan, η οποία μετρά το άθροισμα των οριζόντιων και κάθετων αποστάσεων κάθε πλακιδίου από τη θέση στόχου του, είναι αποδεκτή αλλά όχι πάντα ακριβής. Μπορεί να υποτιμήσει το πραγματικό κόστος για την επίτευξη της κατάστασης στόχου, επειδή δεν λαμβάνει υπόψη αποκλεισμένες ή μη-προσβάσιμες διαδρομές. Σε αυτή την περίπτωση, αυτή η ευρετική δίνει προτεραιότητα στην εξερεύνηση των καταστάσεων που είναι πιο κοντά από την άποψη της απόστασης του Μανχάταν αλλά δεν βρίσκονται στη βέλτιστη διαδρομή, οδηγώντας σε μη βέλτιστες λύσεις.

δ) Greedy, out-of-place

Όπως έχουμε περιγράψει και παραπάνω το αλγόριθμο Greedy, τα ίδια ισχύουν και εδώ.

Σχετικά, με την ευρετική συμπεριφορά, μετράει τον αριθμό των πλακιδίων που δεν βρίσκονται στη σωστή τους θέση στην τρέχουσα κατάσταση σε σύγκριση με την κατάσταση στόχου. Αν και είναι αποδεκτό (που σημαίνει ότι δεν υπερεκτιμά ποτέ το πραγματικό κόστος για τον στόχο), μπορεί μερικές φορές να είναι ανακριβές, οδηγώντας σε μη βέλτιστες λύσεις. Όπως έχουμε και σε αυτήν την περίπτωση.

Λαμβάνει υπόψη μόνο τον αριθμό των πλακιδίων που βρίσκονται σε λάθος θέσεις και δεν λαμβάνει υπόψη το πραγματικό κόστος ή την απόσταση μεταξύ των πλακιδίων και των θέσεων στόχου τους. Αυτή η έλλειψη λεπτομερών πληροφοριών οδηγεί τον αλγόριθμο αναζήτησης στη λήψη αποφάσεων με βάση ένα σχετικά μικρό μέρος της συνολικής δομής του παζλ. Με αποτέλεσμα να γίνεται μεγαλύτερο το μήκος λύσης ώστε να καταγράψει κάθε μικρό μέρος.

3) Σύμφωνα με το παράδειγμα που έχουμε σε αυτό το μέρος της εργασίας, μπορούμε να παρατηρήσουμε ότι αν και όλοι οι αλγόριθμοι δεν έχουν μία βέλτιστη λύση καταφέρνουν να είναι πλήρης. Εκτός των αλγόριθμων DFS και IDS.

Ο αλγόριθμος IDS παρόλο που δεν τερματίζει στο παράδειγμα μας, γνωρίζουμε ότι θα τερματίσει κάποτε αν και πάρει πάρα πολύ χρόνο. Ο συγκεκριμένος αλγόριθμος έχει την ιδιότητα χρήσης των αλγόριθμων BFS και DFS που σημαίνει ότι ψάχνει και στα δύο επίπεδα ώστε να βρει κάποια λύση.

Άρα, καταλαβαίνουμε ότι ο IDS είναι πλήρης αν έχουμε μη-άπειρα δεδομένα όπως και στο παράδειγμα μας παζλ.

Από την άλλη ο DFS δεν τερματίζει στο παραπάνω πίνακα μας. Που πράγματι αυτό ισχύει. Με δεδομένου ότι ψάχνει σε ένα επίπεδο, με αποτέλεσμα να έχει την δυνατότητα να οδηγήσει το αλγόριθμο σε μία κατεύθυνση που ουσιαστικά είναι αδιέξοδο.

Παρόλο που στην άσκηση μας ο Greedy αλγόριθμος καταφέρνει να τερματίσει, δεν είναι πάντοτε πλήρης για κάθε άλλο πρόβλημα. Διότι έχει την δυνατότητα να οδηγήσει σε αδιέξοδο επειδή έχει το χαρακτηριστικό “άπληστο” με αποτέλεσμα να παίρνει λάθος αποφάσεις.

4.) Οι ταχύτεροι αλγόριθμοι είναι αυτή που έχουν το λιγότερο κόστος που μπορούμε να το παρατηρήσουμε και στο πίνακα παραπάνω. Φυσικά, ανάλογα με το αλγόριθμο έχουν διαφορές στις ποιότητες λύσεις που κάθε βήμα που κάνουν χαμηλώνει την ποιότητα του αλγορίθμου.

Ο ταχύτερος αλλά και ο βέλτιστος ταυτόχρονα αλγόριθμός είναι ο Greedy, Euclidean παρόλο που ο Greedy αλγόριθμος δεν είναι σε κάθε περίπτωση ο ταχύτερος και ο βέλτιστος λόγω της ευρετικής Euclidean καταφέρνει να είναι αποδοτικός. Παρόλο αυτό, θεωρητικά δεν ισχύει ότι θα είναι πάντα ο ποιο αποδοτικός. Σε αυτή την περίπτωση απλά είμαστε τυχερή που βρίσκει την λύση σε πολύ καλή απόδοση λόγω της τοποθέτησης των πλακιδίων στο Goal State.

Ο αλγόριθμος A^* και σε κάθε ευρετική που χρησιμοποιεί καταφέρνει να είναι βέλτιστος αν και σε κάποιες περιπτώσεις χρειάστηκε περισσότερα βήματα. Ο λόγος που καταφέρνει να κάνει αυτό είναι επειδή διασφαλίζει τη βέλτιστη λύση, η αναζήτηση A^* , η οποία συνδυάζει ευρετικά με πληροφορίες κόστους αναζητά κάθε περίπτωση για να εγγυηθεί την εύρεση της βέλτιστης λύσης παραμένει και αποδεκτή.

Συμπερασματικά, ο A^* είναι διασφαλιζόμενος αλγόριθμος που είναι βέλτιστος σε κάθε ευρετική.

5) Σύμφωνα με τα αποτελέσματα που διαθέτουμε από το πείραμα μας, παρατηρούμε ότι και οι δύο αλγόριθμοι τερματίζουν σε κάθε ευρετική αναζήτηση. Σχετικά με την βέλτιστη λύση ο A^* σε κάθε ευρετική αναζήτηση διαθέτει το μικρότερο μήκος ενώ στην περίπτωση Greedy μόνο στην Euclidean έχουμε βέλτιστη λύση παρόλο που είναι μία μεθοδολογία που δεν ψάχνει αποδοτικά.

Είναι σαφές που ο Greedy, Manhattan και ο Greedy, out-of-place, δεν έχουν τις βέλτιστες λύσεις με λόγο του Greedy αλγορίθμου να μην είναι optimal. Η ευρετική αναζήτηση Manhattan είναι μια αξιοπρεπής ευρετική, μερικές φορές μπορεί να υπερεκτιμήσει το πραγματικό κόστος για την επίτευξη του στόχου. Αυτή η υπερεκτίμηση μπορεί να οδηγήσει στον αλγόριθμο να κάνει μη βέλτιστες επιλογές, με αποτέλεσμα μια μακρύτερη διαδρομή όπως γίνεται και στο δικό μας παράδειγμα.

Αν και η out-of-place είναι μία απλή ευρετική, δεν είναι πολύ κατατοπιστική επειδή δεν λαμβάνει υπόψη τις σχετικές αποστάσεις μεταξύ των πλακιδίων αλλά και την μνήμη που δεν έχει μία ανησυχία για το χώρο που θα χρειαστεί για ολοκλήρωση. Ως αποτέλεσμα, μπορεί να οδηγήσει σε μη βέλτιστες αποφάσεις για την επιλογή της επόμενης κίνησης.

Ενώ στην περίπτωση A^* ο αλγόριθμος είναι πάντα βέλτιστος αν $h(x) \leq \text{cost-to-goal}$, δηλαδή για να έχουμε μία βέλτιστη λύση στο A^* αρκεί η ευρετική αναζήτηση να είναι μικρότερου

από το κόστος ολοκλήρωσης του αλγορίθμου. Που μπορούμε να το παρατηρήσουμε και στο πίνακα. Με αποτέλεσμα να έχουμε βέλτιστο αλγόριθμο σε κάθε A^* .

6) Η ταχύτητα μπορεί να υπολογιστεί στο συγκεκριμένο μας πείραμα, από το κόστος δηλαδή, το αριθμό κόμβων που έχουν επεκταθεί. Αρχικά, από πρώτη ματιά είναι φανερό ότι οι ευρετικές αναζητήσεις με μικρότερο μήκος λύσεις έχουν και λιγότερο κόστος, με αποτέλεσμα να τους κάνει ταχύτερους για την επίλυση του προβλήματος.

Αναλόγως με την ευρετική αναζήτηση που βρίσκονται οι αλγόριθμοι επηρεάζονται σε σχέση ταχύτητας και ποιότητας. Οι βέλτιστες λύσεις έχουν και την καλύτερη ποιότητα είναι ταχύτερες ενώ αυτές που δεν είναι βέλτιστες έχουν την χειρότερες ποιότητες και ταχύτητες για εύρεση αποτελεσμάτων.

Βέβαια, κάθε ευρετική και αλγόριθμος έχουν μία διαφορετική φιλοσοφία ακόμα και να είναι ταχύτερες, με αποτέλεσμα να έχουν κάνει περισσότερα βήματα λόγω της λειτουργικότητας. Για παράδειγμα, παρόλο που το Greedy, Euclidean και A^* , Euclidean έχουν βέλτιστη λύση έχουν άλλο κόστος λόγω διαφορετικών τρόπων εύρεσης λύσεων των αλγόριθμων. Ένα άλλο παράδειγμα το A^* , Manhattan και A^* , out-of-place αυτή την φορά έχουν ίδιους αλγόριθμους ενώ διαφορετικές ευρετικές, που και αυτό τους οδηγεί σε διαφορετικούς κόστους λόγω τις φιλοσοφίες των ευρετικών λειτουργικών.

Μέρος Β: Κύβος του Ρούμπικ (mini-cube)

1.) Το μέγεθος του χώρου κατάστασης ενός κύβου του Ρούμπικ 2x2 είναι 3.674.160. Ουσιαστικά το μέγεθος του χώρου κατάστασης είναι όλες οι πιθανές τροποποιήσεις που μπορούμε να κάνουμε σε ένα πεδίο.

Που το βρίσκουμε από το τύπο:

$$\frac{8! \times 3^7}{24} = 7! \times 3^6 = 3,674,160.$$

2.) Σύμφωνα, με τα πειράματα που έχουμε πραγματοποιήσει παρατηρούμε διαφορές σε κάθε αλγόριθμο ανάλογα με τα βήματα που χρειάστηκαν για να ανακατευθεί ο κύβος.

α) Iterative Deepening Depth-First Search (IDDFS)

Το Iterative Deepening Depth-First Search (IDDFS) είναι ένας αλγόριθμος αναζήτησης που μπορεί να βρει τις βέλτιστες λύσεις για τον κύβο του Rubik 2x2, δεδομένου του χρόνου και των χωρών. Είναι ιδιαίτερα κατάλληλο για την επίλυση τέτοιων προβλημάτων και αναζήτηση προβλημάτων, συμπεριλαμβανομένου του κύβου του Ρούμπικ, όπου ο χώρος κατάστασης είναι σχετικά μικρός και η βέλτιστη λύση είναι εγγυημένη ότι υπάρχει.

Το IDDFS λειτουργεί εμβαθύνοντας επαναληπτικά το βάθος του δέντρου αναζήτησης, εξερευνώντας το χώρο αναζήτησης σε ένα συγκεκριμένο βάθος και στη συνέχεια αυξάνοντας σταδιακά το όριο βάθους με κάθε επανάληψη μέχρι να βρει μια λύση. Δεδομένου ότι εξερευνά ολόκληρο τον χώρο αναζήτησης συστηματικά, είναι εγγυημένο ότι

θα βρει τη βέλτιστη λύση εάν υπάρχει. Το βάθος στο οποίο βρίσκει μια λύση θα είναι το μήκος της βέλτιστης λύσης ως προς τον αριθμό των κινήσεων.

β) Iterative-deeping A* (IDA*)

Ο IDA* είναι ένας αλγόριθμος αναζήτησης που συνδυάζει χαρακτηριστικά τόσο της αναζήτησης πρώτα σε βάθος (DFS) όσο και του A*. Μπορεί να χρησιμοποιηθεί για την εύρεση βέλτιστων λύσεων για τον κύβο του Ρούμπικ 2x2 αναζητώντας επαναληπτικά τον χώρο λύσης, αυξάνοντας το όριο βάθους με κάθε επανάληψη μέχρι να βρει τη βέλτιστη λύση. Το IDA* είναι εγγυημένο ότι βρίσκει βέλτιστες λύσεις, αλλά μπορεί να μην είναι πάντα η πιο αποτελεσματική επιλογή για μικρότερους κύβους λόγω της διαθεσιμότητας προυπολογισμένων βέλτιστων λύσεων όπως μπορούμε να παρατηρήσουμε και από την ιστοσελίδα.

γ) Simulated Annealing (SA)

Ο Simulated Annealing (SA) είναι ένας ευρετικός αλγόριθμος αναζήτησης που χρησιμοποιείται συχνά για προβλήματα βελτιστοποίησης. Ενώ μπορεί να βρει αρκετά καλές λύσεις για τον κύβο του Rubik 2x2, δεν εγγυάται η βέλτιστη. Είναι ένας στοχαστικός αλγόριθμος (δηλαδή μη-ντετερμινιστικός που συμπεριφέρεται με πιθανότητες) που μπορεί να μην βρίσκει πάντα τη συντομότερη δυνατή λύση. Άρα θα μπορούσαμε να πούμε ότι δεν είναι ούτε και ο πιο αξιόπιστος.

δ) Genetic Algorithm (GA)

Ο Genetic Algorithm είναι μια μορφή εξελικτικού υπολογισμού και χρησιμοποιούνται πιο συχνά για προβλήματα βελτιστοποίησης. Μπορούν να βρουν αρκετά καλές λύσεις για τον κύβο Rubik 2x2, αλλά, όπως και το SA, δεν εγγυάται η βέλτιστη. Ο Genetic Algorithm εξερευνάει έναν πληθυσμό πιθανών λύσεων και η ποιότητα των λύσεων μπορεί να βελτιωθεί με την πάροδο του χρόνου, αλλά μπορεί να μην βρίσκουν πάντα τη συντομότερη διαδρομή λόγω των βημάτων που θα χρειαστεί για βελτιστοποίηση.

ε) Weighted A* (WA*)

Ο A* με κατάλληλο ευρετικό μπορεί να χρησιμοποιηθεί για την εύρεση βέλτιστων λύσεων για τον κύβο του Ρούμπικ 2x2. Το Weighted A* εστιάζει την προσπάθειά του αναζήτησής του σε καταστάσεις με χαμηλές ευρετικές τιμές, αλλά δεν αγνοεί εντελώς το g-cost. Ως αποτέλεσμα, το Weighted A* ξεκινά με μια πολύ πιο μικρή επίπεδη αναζήτηση από άλλους αλγόριθμους. Για αυτό το λόγο επειδή προσπαθεί να λύσει το πρόβλημα με χαμηλές ευρετικές τιμές, η αύξηση του βάρους κάνει τον αλγόριθμο να δίνει προτεραιότητα στην ταχύτητα έναντι της βέλτιστης, ενώ η μείωση του βάρους δίνει προτεραιότητα στη βέλτιστη έναντι της ταχύτητας. Άρα στο κύβο του Rubik όταν εφαρμοστεί αυτό το χαρακτηριστικό του αλγόριθμου ή θα προσπαθεί να βρει κάποια γρήγορη λύση χωρίς να προσπαθεί να βρει πιο αξιόπιστο ή το αντίθετο. Με αποτέλεσμα να μην βρίσκει την βέλτιστη λύση.

στ) Bidirectional BFS (BiBFS)

Το Bidirectional BFS είναι ένας αλγόριθμος αναζήτησης που εξερευνά τον χώρο αναζήτησης τόσο από την αρχή όσο και από την κατάσταση στόχου ταυτόχρονα. Παρόλο που μπορεί να είναι αποτελεσματικό και να εγγυηθεί τη βέλτιστη απόδοση για ορισμένα προβλήματα, συμπεριλαμβανομένου του κύβου Rubik 2x2, μπορεί να μην είναι πάντα η πιο αποτελεσματική επιλογή για μικρότερους κύβους λόγω της διαθεσιμότητας

προϋπολογισμένων βέλτιστων λύσεων. Αλλά θα ήταν μία καλή επιλογή σε εύρεση τέτοιων προβλημάτων.

3.) Σύμφωνα και με τις εξομοιώσεις που έχουμε κάνει μπορούμε να παρατηρήσουμε ότι κάθε αλγόριθμος έστω και μία φορά τερματίζει. Αλλά έχουμε και περιπτώσεις που αλγόριθμος δεν τερματίζει. Ναι μεν μπορούσε να λύση κάποια ακολουθία αλλά άλλη μία όχι.

Θα περιγράψουμε αυτούς τους αλγορίθμους σε λίστα όπως κάναμε και στο 2^ο ερώτημα.

α) Iterative Deepening Depth-First Search (IDDFS)

Για το συγκεκριμένο αλγόριθμο, αφού δεν έχουμε καταστάσεις μη-αριθμήσιμα σημαίνει ότι είναι πλήρης και ο αλγόριθμος. Βρίσκει πάντα κάποιο αποτέλεσμα με δεδομένου που προσπαθεί να λύση το πρόβλημα σε δύο διαστάσεις. Άρα το ίδιο θα ισχύει και για το κύβο του Rubik με αποτέλεσμα να τερματίσει.

β) Iterative-deeping A* (IDA*)

Ένας άλλος πλήρης αλγόριθμος αναζήτησης. Εξερευνά συστηματικά τον χώρο για να βρει μια λύση εάν υπάρχει. Άρα είναι ένας πλήρης αλγόριθμος που βρίσκει κάποιο λύση πάντα λόγω το συστηματικό χαρακτήρα που παρέχει.

γ) Simulated Annealing (SA)

Δεν είναι απαραίτητα πλήρης. Είναι ένας στοχαστικός αλγόριθμος, (μη-ντετερμινιστικός και συμπεριφέρεται με πιθανότητες) μπορεί να μην βρίσκει πάντα μια λύση ακόμα κι αν υπάρχει. Είναι πιο κατάλληλο για προβλήματα βελτιστοποίησης παρά για εγγυημένη εύρεση λύσεων όπως το κύβο του Rubik.

δ) Genetic Algorithm (GA)

Ο Genetic Algorithm είναι αλγόριθμος εξέλιξης. Που σημαίνει ότι μπορεί να είναι πλήρης σε κάποιες περιπτώσεις ανάλογα της εξέλιξης και την δυσκολία του κύβου. Αλλά επειδή η εξέλιξη δεν σημαίνει ότι θα είναι προς την σωστή κατεύθυνση, θα υπάρξουν και προβλήματα που δεν είναι πλήρης. Άρα ο αλγόριθμος Genetic δεν είναι πλήρης.

ε) Weighted A* (WA*)

Είναι πλήρης. Ο A* είναι ένας πλήρης αλγόριθμος όταν χρησιμοποιείται με μια αποδεκτή και συνεπή ευρετική συνάρτηση. Παρόλο που το weighted A* έχει μία πιο επίπεδη προσέγγιση αυτό δεν σημαίνει ότι δεν είναι πλήρης και βρίσκει το αποτέλεσμα. Δηλαδή, για καλύτερη εξήγηση το Weighted A* έχει την εξίσωση $f = g + eh$ όπου $e > 1$ για να είναι πλήρης. Αλλιώς, είναι προφανές ότι θα εγκλωβιστεί.

στ) Bidirectional BFS (BiBFS)

Το Bidirectional BFS είναι πλήρες για την εύρεση λύσης μεταξύ δύο δεδομένων καταστάσεων (κατάσταση έναρξης και στόχος). Παρόλο που δεν είναι σίγουρο αν θα εξερευνήσει κάθε κατάσταση θα φτάσει σε ένα τελικό αποτέλεσμα αφού θα ψάχνει μέχρι βρει το αποτέλεσμα στο πλάτος.

4)

Αλγόριθμος	Μήκος Λύσης	Είναι Βέλτιστη;	Ποσοστό απόκλισης	Χρόνος
IDDFS	11	ΝΑΙ	0%	107,6s
IDA*	11	ΝΑΙ	0%	84,4s
SA	11	ΝΑΙ	0%	97,1s
GA	19	ΟΧΙ	35,71%	12,7s
WA*	17	ΟΧΙ	21.42%	1,08s
BiBFS	11	ΝΑΙ	0%	193,3ms

Τα βήματα που έχουμε κάνει για το συγκεκριμένο πείραμα

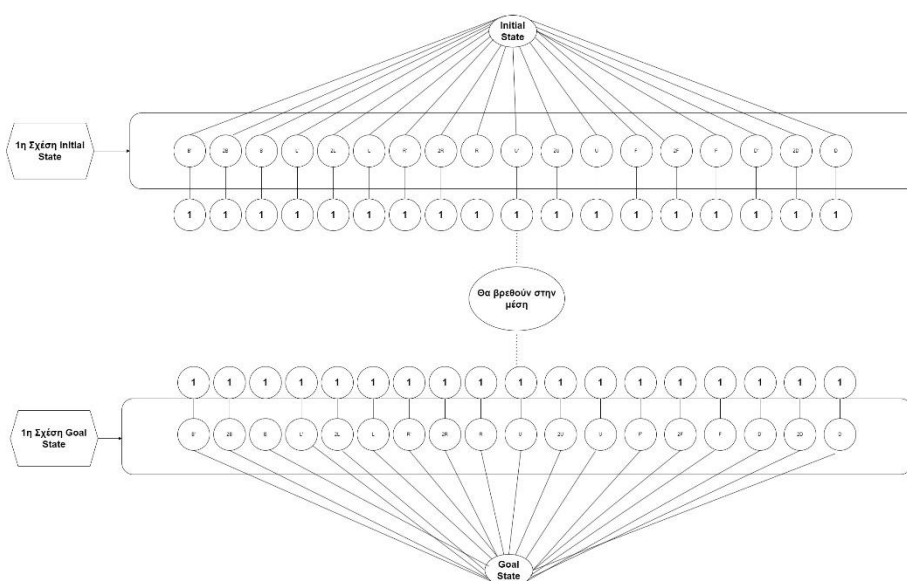
31: B' U' R B D U' R' B' F' R' D U' D' R' D B F B' D' R 2L U D' L'
U' 2R D' L F

5) α.)BiBFS

Σύμφωνα με την εκφώνηση που έχουμε μας ζητιέται να κάνουμε ένα δένδρο σχετικό με το κύβο του 2^{ου} μέρους της άσκησης. Ο αλγόριθμος BiBFS όπως γνωρίζουμε είναι προς 2 κατευθύνσεις. Δηλαδή εξερευνάει από το Initial State αλλά και από το Goal State.

Θα χρειαστεί να σημειώσουμε ότι ο αλγόριθμος bidirectional που παρέχουμε είναι σε μορφή BFS. Δηλαδή, αυτό σημαίνει ότι θα αναζητάμε πρώτα προς το πλάτος.

Για μια γενική εικόνα έχουμε την συγκεκριμένη προσέγγιση.



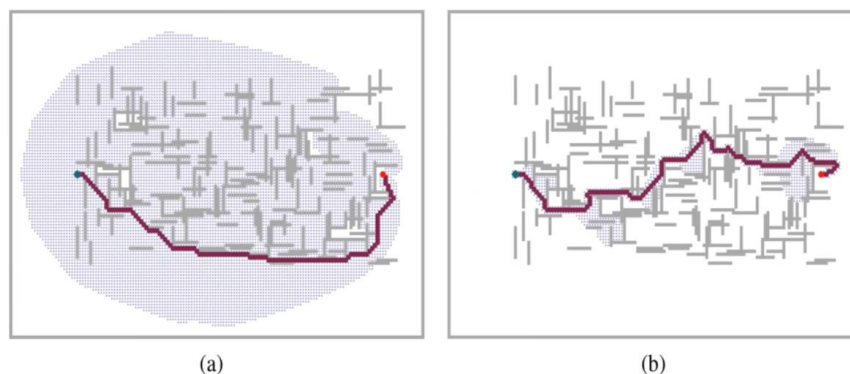
Σε αυτή την γενική περίπτωση έχουμε τα Goal και Initial states. Αρχικά, ο κύβος θα διαλέξει έναν κόμβο που του δίνεται και θα αναζητήσει προς το πλάτος αν υπάρχει και άλλος κόμβος. Αυτό θα το κάνει και από τις δύο κατευθύνσεις.

Στην συνέχεια, όταν εξερευνηθεί η πρώτη σχέση και των 2 κατευθύνσεων, εφόσον δεν υπάρχει κάποιος άλλος κόμβος στο πλάτος, θα επεκταθεί ο αλγόριθμος προς τα μέσα. Έχοντας την ίδια πιθανότητα να ξανασυναντήσουμε κάποιο κόμβο ή κόμβους από τα 18 διαθέσιμα. Που η ιδεολογία συνεχίζεται όπως και στις μεταβάσεις προς τα μέσα.

Με αυτή την λογική ο αλγόριθμος θα κλείσει προς τα μέσα και θα βρεθούν στο στην μέση το Initial και Goal State.

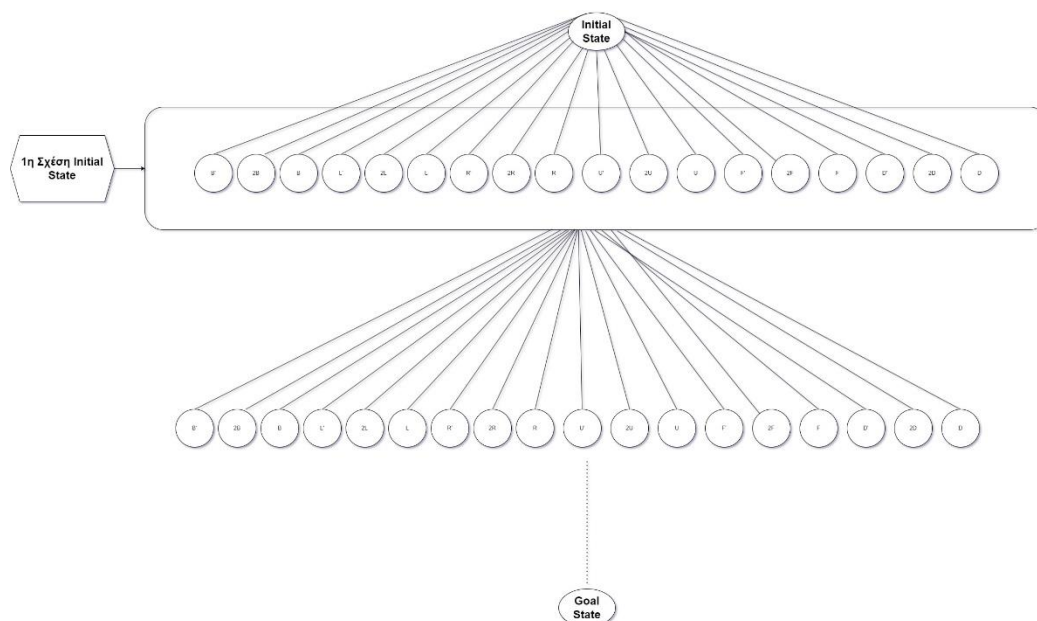
β) WA*

Όπως γνωρίζουμε και από την θεωρία το WA* δεν ξοδεύει κόστος όσο το A*. Έχει μία συμπεριφορά με ιδιότητα να προσπαθεί να βρει την λύση με λιγότερο κόστος που μπορεί.



Για καλύτερη κατανόηση, ο (a) που είναι το A* εξερευνάει περισσότερο πεδίο για να βρει ένα μονοπάτι, ενώ το (b) που είναι το Weighted A* προσέχει το κόστος και δεν εξερευνάει κάθε πεδίο στο παράδειγμα.

Που στο δένδρο που θα περιγράψουμε θα ισχύουν τα ίδια:



Αρχικά έχουμε μία αρχική κατάσταση και επιλέγουμε ανάλογα με το μικρότερο κόστος κάθε μετάβασης $g(n)$. Όπως μπορούμε να παρατηρήσουμε και από την πρώτη σχέση έχουμε κάθε πιθανή μετάβαση.

Που αυτό συνεχίζεται και από κάτω μέχρι να φτάσουμε στο Goal State.

Σύμφωνα και με την εκφώνηση, έχουμε $50 \cdot h(n)$ στις προηγούμενες με αποτέλεσμα να έχουμε την συμπεριφορά $g(n) + (50 \cdot h(n))$. Όπου το $g(n)$ αρχικά είναι 1 και αυξάνεται όταν πέφτουμε επίπεδο αφού θα χρειαστούμε 2 κινήσεις για να πάμε στην δεύτερη κατάσταση από την αρχική κατάσταση. Άρα σε κάθε επίπεδο αυξάνεται το $g(n)$.

6) Όπως μπορούμε να δούμε και στο πίνακα που έχουμε αναφερθεί για τους αλγόριθμους και τα δεδομένα τους για κάθε ένα χαρακτηριστικό που μας ζητείται, οι αλγόριθμοι IDDFS, IDA*, SA και BiBFS έχουν μία λύση που είναι βέλτιστη. Που ισχύει ότι είναι βέλτιστες αλλά και θεωρητικά.

Παρόλο που αυτοί οι αλγόριθμοι είναι βέλτιστες δεν σημαίνει ότι πάντα θα έχουμε την καλύτερη απόδοση. Αν πάρουμε μόνο τους αλγόριθμους που είναι βέλτιστες, ο BiBFS έχει την καλύτερη απόδοση σε σχέση χρόνου και λύσης. Οι άλλοι αλγόριθμοι ναι μεν έχουν μία βέλτιστη λύση, δεν έχουν το καλύτερο χρόνο για να βρουν την λύση που σημαίνει ότι οι άλλοι αλγόριθμοι που έχουν περισσότερα βήματα για να μπορέσουν να λύσουν το πρόβλημα ενώ δεν είναι και βέλτιστη, μας παίρνουν λιγότερο χρόνο για να βρούμε το αποτέλεσμα.

Που σαν απόδοση, ειδικά σε μεγαλύτερα προβλήματα μπορούν να είναι χρήσιμα από βέλτιστους αλγόριθμους. Που το ποσοστό απόκλισης δεν είναι και το μεγαλύτερο. Φυσικά σε προβλήματα μικρού μέγεθος δεν έχουν μεγάλη χρησιμότητα λόγω των βημάτων αλλά και του χρόνου να είναι μικρό. Εν τέλει, η διαφορά δεν είναι μεγάλη που τους κάνει χρήσιμους σε μεγαλύτερα προβλήματα αλλά και για το λόγο ποσοστό απόκλισης που δεν είναι πολύ μεγάλος.

Βέβαια, θα χρειαστεί να αναφερθούμε ότι σε μεγαλύτερα προβλήματα αυτό το ποσοστό απόκλισης θα γίνει πιο φανερό. Αλλά παρόλο αυτό η χρήση του δεν θα ήταν άχρηστή λόγω του μικρότερου χρόνου από τους βέλτιστους αλγόριθμους εκτός του BiBFS. Ειδικά ο αλγόριθμος WA*.

Ο αλγόριθμος BiBFS είναι η καλύτερη επιλογή αλγορίθμου σε μικρά ή μεγάλα προβλήματα αρκεί να έχουμε μη-άπειρα βήματα. Λόγω του χαρακτηριστικό που αναζητά από την αρχικό αλλά και τελικό κόμβο το κάνει πάρα πολύ γρήγορο διότι κερδίζει χρόνο αναζητώντας και από τις δύο κατευθύνσεις. Πράγμα που άλλοι αλγόριθμοι θα έψαχναν με διαφορετική συμπεριφορά. Καταφέρνει να βρει και την βέλτιστη λύση αφού χρησιμοποιείται για αναζήτηση μονοπατιών που έχουν ενιαίο κόστος. Τέλος, μπορούμε να επιβεβαιώσουμε ότι ο BiBFS έχει την καλύτερη απόδοση.

7) Όπως μπορούμε να παρατηρήσουμε και στο πίνακα παραπάνω οι αλγόριθμοι IDA* και WA* έχουν 11 και 17 μήκος λύσης αντίστοιχα.

Επειδή και οι δύο αλγόριθμοι χρησιμοποιούν την ευρετική A^* που είναι από της καλύτερης στην εύρεση της βέλτιστης λύσης, θα περιμέναμε να έχουμε και στο WA* μία βέλτιστη λύση.

Μια βέλτιστη διαδρομή ελαχιστοποιεί το συνολικό κόστος ενώ ο Weighted A* έχει την προσέγγιση να μειώνει το κόστος της διαδρομής που έχουμε εξερευνήσει προηγουμένως. Αυτή η διαφορά σημαίνει ότι η σειρά των κόμβων στην ουρά προτεραιότητας δεν είναι πλέον η καλύτερη υπόθεση για μια βέλτιστη διαδρομή, πράγμα που σημαίνει ότι ορισμένα μη βέλτιστα μονοπάτια μπορούν να επισκεφθούν πριν από τη βέλτιστη διαδρομή. Ουσιαστικά ο Weighted A* ψάχνοντας να βρει την βέλτιστή λύση καταστρέφει την βέλτιστη διαδρομή λόγω αναζήτησης.

Ενώ ο IDA* που είναι μία παραλλαγή του αλγορίθμου Iterative Deepening Depth-First Search που δανειζεται την ιδέα να χρησιμοποιηθεί μια ευρετική συνάρτηση για να εκτιμήσει συντηρητικά το υπόλοιπο κόστος για να φτάσουμε στον στόχο από τον αλγόριθμο αναζήτησης A*. Αναζητά σε 2 επίπεδα (DFS και BFS) καταφέρνει να βρει την βέλτιστη λύση αφού έχει 2 τρόπους εύρεσης, εν τέλη και την βέλτιστη λύση.

Για κύβους μεγαλύτερης διάστασης, που έχει περισσότερες πιθανές καταστάσεις και πολύπλοκες διαδρομές λύσης από τον κύβο 2x2. Το Weighted A* μπορεί να είναι πιο χρήσιμο σε αυτό το πλαίσιο, εάν θέλουμε να ισορροπήσουμε μεταξύ της γρήγορης εύρεσης λύσεων και της εύρεσης λύσεων που είναι πιο κοντά στη βέλτιστη συντομότερη διαδρομή. Η επιλογή βάρους στο Weighted A* μας βοηθά να ρυθμίσουμε με ακρίβεια τη συμπεριφορά του αλγορίθμου για να επιτυγχάνουμε μια ισορροπία μεταξύ της ποιότητας της λύσης και της υπολογιστικής απόδοσης (χρόνος). Άρα επιλέγοντας το Weighted A* και διαμορφώνοντας με μεγαλύτερο βάρος μπορεί να μας δώσει πιο αποδοτική λύση από αυτά που θα έδινε στο IDA* λόγω του χρόνου.

Σχετικά με την IDA*, θα μας έδινε πάλι την βέλτιστη λύση αν είχαμε κύβο με μεγαλύτερες διαστάσεις. Αλλά όπως μπορούμε να παρατηρήσουμε και από το πίνακα η διαδικασία θα γινόταν ακόμα πιο χρονοβόρα αφού και στο κύβο 2x2 μας πήρε αρκετό χρόνο για να βρει την βέλτιστη λύση. Με αποτέλεσμα το WA* να είναι πιο αποδοτικό σε μεγαλύτερους κύβους λόγω το ότι είναι γρήγορος παρόλο που δεν βρίσκει την βέλτιστη λύση.

8) Η παραδεκτή ευρετική που θα μπορούσαμε να προτείνουμε είναι η Manhattan. Για να βρούμε την απόσταση μεταξύ τις αρχικής και τελικής κατάστασης, θα χρειαστούμε να υπολογίσουμε την απόσταση πρώτα της αρχικής και τελικής. Στην συνέχεια της αρχικής+1 και τελικής και αυτό θα διαιρείται δια 8. Δηλαδή, σε κάθε βήμα υπολογίζεται η απόσταση των λάθος αυτοκόλλητων, θα προστίθενται μεταξύ τους και θα διαιρούνται δια 8, αφού σε κάθε κίνηση μετακινούνται 8 αυτοκόλλητα μέχρι την τελική κατάσταση

Η προσέγγιση της Manhattan είναι από τις καλύτερες διότι είναι αποδεκτή, που σημαίνει ότι ποτέ δεν υπερεκτιμά το κόστος επίτευξης του στόχου. Στο πλαίσιο της επίλυσης ενός κύβου Rubik, αυτή η ιδιότητα είναι πολύτιμη επειδή διασφαλίζει ότι η ευρετική καθοδηγεί τον αλγόριθμο αναζήτησης προς τη σωστή κατεύθυνση χωρίς να προτείνει κινήσεις που είναι περιττές ή μη βέλτιστες.